Image Exposure Correction

Che-Hsi Chang, Yu-Tsen Chen, Yu-jung Tseng[1]

[a]*National Tsing Hua University, Department of Electrical Engineering*

Abstract—Low-light and improper exposure correction is a computationally intensive task in image processing, especially for illumination-based methods that rely on global operations and full-figure data access. While existing approaches such as illumination map estimation and dual illumination modeling achieve promising visual results, they are not directly suitable for efficient hardware implementation.
In this project, we implement and optimize an illumination-based exposure correction algorithm with the explicit goal of enabling hardware acceleration. Starting from a Python-based functional implementation, we analyze the computational structure of the algorithm and reformulate it to support image partitioning and localized processing. The computation flow is reorganized to reduce large memory requirements, and a memory layout is carefully designed for intermediate illumination maps and image buffers to improve data reuse and hardware efficiency. Experimental results show that the optimized design preserves the visual quality of illumination-based enhancement while providing a clear pathway toward efficient hardware realization and acceleration.

Keywords—*exposure correction, illumination map estimation, hardware acceleration, image partition*

1. Introduction

Images captured under challenging lighting conditions often suffer from improper exposure, including underexposed, overexposed, or mixed-exposure regions. Such degradation reduces visual quality and may negatively affect subsequent image processing and computer vision tasks. Therefore, exposure correction and low-light enhancement are essential preprocessing steps in modern imaging pipelines, especially when real-time processing or deployment on resource-constrained devices is required.

Among existing approaches, illumination-based methods derived from Retinex theory are attractive due to their interpretability and deterministic structure. By modeling an image as the pixel-wise product of reflectance and illumination, exposure correction can be reformulated as an illumination estimation problem, enabling direct and controllable brightness manipulation without requiring training data. However, many illumination-based solvers are developed under full-frame assumptions and involve global intermediate buffers and bandwidth-intensive memory access, which become major obstacles for efficient hardware acceleration.

In this project, we bridge the gap between illumination-based exposure correction and hardware-oriented implementation. We first reproduce the reference algorithms in Python to validate functional correctness and visual quality, and then reorganize the computation to support partition-based processing. To improve hardware efficiency, we explicitly design the dataflow and memory allocation for intermediate illumination maps and memory buffers, enabling localized data reuse and reduced memory size requirement. The proposed implementation preserves the visual quality of illumination-based enhancement while providing a practical pathway toward scalable hardware realization.

2. Algorithm

Under the Retinex-based image formation model, an observed RGB image can be expressed as

$$I(x) = R(x) \circ T(x), \tag{1}$$

where x denotes the pixel location. Here, $R(x)$ represents the intrinsic reflectance of the scene, which is determined by the physical material properties and surface texture of objects, and $T(x)$ denotes the illumination map describing the spatial distribution of lighting. The operator $\circ$ indicates element-wise multiplication. In this formulation, the observed image $I(x)$ can be interpreted as the appearance of an object under external illumination.

Note: $I(x)$, $R(x)$, and $T(x)$ are all RGB-valued matrices.

Given the observed image $I(x)$ (e.g., a photograph captured by a camera), the objective of illumination-based enhancement is to recover the intrinsic reflectance $R(x)$ by compensating for the illumination component. In practice, jointly estimating both $R(x)$ and $T(x)$ from a single image is challenging and requires additional assumptions. As a result, many illumination-based enhancement methods focus on estimating the illumination map $\hat{T}(x)$, and obtain an approximation of the reflectance through element-wise division,

$$\hat{R}(x) = \frac{I(x)}{\hat{T}(x) + \varepsilon}, \tag{2}$$

where ε is a small constant introduced to ensure numerical stability. Although $\hat{R}(x)$ is not guaranteed to be a physically exact reflectance, it serves as an illumination-corrected image that approximates the intrinsic appearance of the scene.

Note: The **hat** sign here (ex: $\hat{R}(x)$, and $\hat{T}(x)$) represent the estimation result.

2.1. Initial illumination map

To recover the illumination-corrected image $\hat{R}(x)$, an initial estimation of the illumination map $\hat{T}(x)$ is required. Following common practice in illumination-based enhancement, we adopt the Max-RGB strategy to obtain a pixel-wise initialization:

$$\hat{T}_i(x) = \max\{I_R(x), I_G(x), I_B(x)\}. \tag{3}$$

This formulation assumes that the illumination at each pixel is no weaker than the strongest color-channel response, providing a simple and robust initialization while avoiding numerical instability during subsequent division. However, since the estimation is purely local, reflectance-related textures may be embedded into the initial illumination map. As illumination is expected to vary smoothly across space, further refinement is required to suppress high-frequency artifacts and recover a structure-consistent illumination distribution.

2.2. LIME estimation

To refine the initial illumination estimate obtained by the Max-RGB strategy, LIME introduces a structure-aware optimization framework that explicitly enforces desirable properties of illumination. The refinement process aims to remove reflectance-related texture from the illumination map while preserving major illumination structures, such as shadows and smooth lighting transitions.

Specifically, the refined illumination map $T(x)$ is obtained by solving the following optimization problem:

$$\min_{T} \ \|T - \hat{T}\|_F^2 + \alpha \|W \circ \nabla T\|_1, \tag{4}$$

Note: The above function gives as scaler.

where $\hat{T}(x)$ denotes the initial illumination estimate obtained from the Max-RGB operation, $\|\cdot\|_F$ represents the Frobenius norm, ∇T denotes the spatial gradient of the illumination map, and $\circ$ indicates element-wise multiplication. The parameter α controls the trade-off between fidelity to the initial illumination estimate and the smoothness of the refined illumination map.

The first term enforces consistency between the refined illumination $T(x)$ and the initial estimate $\hat{T}(x)$, ensuring that the overall brightness distribution is preserved. The second term penalizes large

spatial gradients in the illumination map, encouraging spatial smoothness. To avoid over-smoothing important illumination boundaries, a structure-aware weighting matrix W is introduced. This weighting suppresses gradients in textured regions while allowing significant gradients at major illumination transitions, thereby preserving perceptually meaningful lighting structures.

$$W_h(x) \leftarrow \frac{1}{\left|\nabla_h \hat{T}(x)\right| + \varepsilon}, \qquad W_v(x) \leftarrow \frac{1}{\left|\nabla_v \hat{T}(x)\right| + \varepsilon} \tag{5}$$

The illumination refinement is formulated as a scalar optimization problem with respect to the illumination map. However, the objective function contains both a quadratic fidelity term and an ℓ_1-norm regularization on the spatial gradients, which is not directly solvable in a closed form. To handle this formulation efficiently, LIME adopts the Augmented Lagrangian Method (ALM), which enables the optimization to be decomposed into a sequence of simpler subproblems that can be solved alternately.

$$\min_{T,G} \|T - \hat{T}\|_F^2 + \alpha \|W \circ G\|_1, \quad \text{s.t. } G = \nabla T \tag{6}$$

The augmented Lagrangian function of (6) can be written in the following

$$\mathcal{L}(T, G, Z) = \|T - \hat{T}\|_F^2 + \alpha \|W \circ G\|_1 + \langle Z, \nabla T - G \rangle + \frac{\mu}{2} \|\nabla T - G\|_F^2 \tag{7}$$

where $\langle \ , \ \rangle$ represents matrix inner product, μ is a positive penalty scalar, and Z is the Lagrangian multiplier. There are three variables, including T, G and Z to solve.

2.3. ALM Solver

The Augmented Lagrangian Method (ALM) solves the illumination refinement problem by iteratively minimizing the augmented Lagrangian with respect to three variables, namely T, G, and Z. At each iteration, the update rules are obtained from the first-order optimality conditions of $\mathcal{L}(T, G, Z)$, i.e.,

$$\frac{\partial \mathcal{L}}{\partial T} = 0, \qquad 0 \in \frac{\partial \mathcal{L}}{\partial G}, \qquad \frac{\partial \mathcal{L}}{\partial Z} = 0, \tag{8}$$

where the second condition is understood as a subgradient optimality condition due to the ℓ_1 term.

T-subproblem (FFT-based solution)

Setting $\partial \mathcal{L} / \partial T = 0$ yields

$$2(T - \hat{T}) + \mu^{(t)} \nabla^\top \left(\nabla T - G^{(t)}\right) + \nabla^\top Z^{(t)} = 0, \tag{9}$$

which can be rearranged into the following linear system:

$$\left(2I + \mu^{(t)} \nabla^\top \nabla\right) T = 2\hat{T} + \mu^{(t)} \nabla^\top \left(G^{(t)} - \frac{Z^{(t)}}{\mu^{(t)}}\right). \tag{10}$$

A key observation is that $\nabla^\top \nabla$ corresponds to a convolutional (circulant) operator under periodic boundary conditions. Therefore, instead of solving (10) by Gaussian elimination, T can be efficiently computed in the frequency domain using FFT/iFFT.

Let D_h and D_v denote the discrete finite-difference filters corresponding to ∇_h and ∇_v, respectively, and let $\mathcal{F}(\cdot)$ and $\mathcal{F}^{-1}(\cdot)$ denote the 2D Fourier transform and inverse Fourier transform. Then the closed-form FFT-based update of T can be written as

$$T^{(t+1)} = \mathcal{F}^{-1}\left(\frac{\mathcal{F}\left(2\hat{T} + \mu^{(t)} \nabla^\top \left(G^{(t)} - \frac{Z^{(t)}}{\mu^{(t)}}\right)\right)}{2 + \mu^{(t)} \left(\left|\mathcal{F}(D_h)\right|^2 + \left|\mathcal{F}(D_v)\right|^2\right)} \right), \tag{11}$$

where the division is performed element-wise in the frequency domain.

G-subproblem (shrinkage)

The update of G admits a closed-form solution via soft-thresholding:

$$G^{(t+1)} = \mathcal{S}_{\frac{\alpha W}{\mu^{(t)}}} \left(\nabla T^{(t+1)} + \frac{Z^{(t)}}{\mu^{(t)}} \right), \tag{12}$$

with the element-wise shrinkage operator defined as

$$\mathcal{S}_\tau(y) = \text{sign}(y) \cdot \max(|y| - \tau, 0). \tag{13}$$

Multiplier and penalty updates

Finally, the Lagrange multiplier and penalty parameter are updated by

$$Z^{(t+1)} = Z^{(t)} + \mu^{(t)} \left(\nabla T^{(t+1)} - G^{(t+1)}\right), \qquad \mu^{(t+1)} = \rho\, \mu^{(t)},\ \rho > 1. \tag{14}$$

By alternately updating T, G, Z, and μ according to the above equations, ALM gradually converges to the refined illumination map $T(x)$.

2.4. Image Recover

After the ALM iterations converge, we obtain the refined illumination map $T(x)$. In practice, $T(x)$ is normalized to $(0, 1)$ to represent relative illumination intensity. To provide an additional knob for controlling the enhancement strength, we apply gamma correction on the illumination map and define the gamma-adjusted illumination

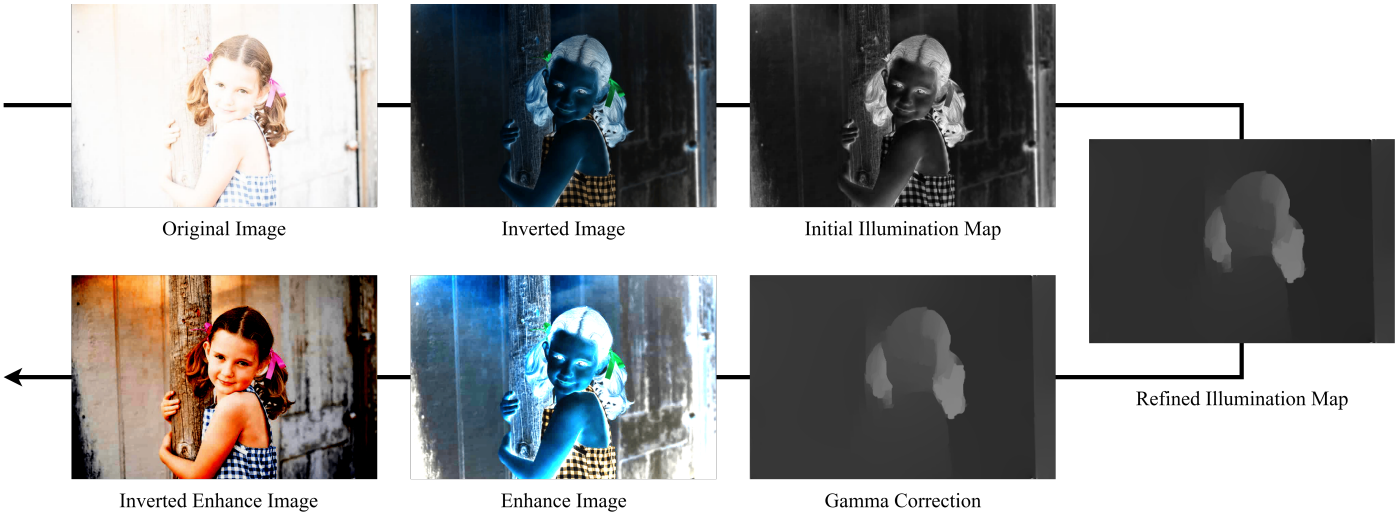

Figure 1. Overexposure image recover process

as

$$\tilde{T}(x) = T(x)^{\gamma}, \tag{15}$$

where γ is a user-defined parameter. Note that since $0 < T(x) < 1$, a smaller γ produces a larger $\tilde{T}(x)$, which increases the divisor and leads to a darker recovered image; conversely, a larger γ decreases $\tilde{T}(x)$ and results in stronger enhancement (a brighter output).

Finally, the illumination-corrected image is recovered by dividing the observed image by the gamma-corrected illumination map:

$$\hat{R}(x) = \frac{I(x)}{\tilde{T}(x)} = \frac{I(x)}{T(x)^{\gamma}}, \tag{16}$$

2.5. Handling over-exposed images by inversion

The LIME pipeline is primarily designed for low-light enhancement, where the key objective is to amplify reflectance by compensating for insufficient illumination. When directly applied to over-exposed images, the Max-RGB initialization and subsequent illumination refinement may become ineffective because saturated regions already approach the upper bound (near 1 after normalization), leaving limited room for meaningful illumination estimation and correction. To extend the method to handle over-exposed cases, we adopt an intensity inversion strategy.

Given a normalized input image $I(x) \in [0, 1]$, we first construct the inverted image

$$I^{\text{inv}}(x) = 1 - I(x), \tag{17}$$

so that over-exposed bright regions in $I(x)$ are mapped to low-intensity regions in $I^{\text{inv}}(x)$. We then apply the same LIME-based illumination estimation and recovery process to obtain an enhanced inverted result $\hat{R}^{\text{inv}}(x)$. Finally, the corrected image in the original intensity domain is reconstructed by inverting back:

$$\hat{R}(x) = 1 - \hat{R}^{\text{inv}}(x). \tag{18}$$

This simple transformation allows the low-light enhancement mechanism of LIME to be reused for over-exposure correction while keeping the overall computation structure unchanged.

3. Implementation

3.1. Motivation of image partitioning

Although the algorithm in Section 2 achieves effective exposure correction under the full-frame setting, a direct hardware implementation is inefficient due to its heavy dependency on global data access and large intermediate buffers. In particular, the ALM-based illumination refinement maintains multiple full-resolution maps (e.g., T, G, and Z) and repeatedly applies structured operators such as $\nabla(\cdot)$, $\nabla^{\top}(\cdot)$, and FFT/iFFT-based solvers. When the input resolution increases, storing and updating these intermediate variables at full frame quickly becomes the dominant cost in terms of SRAM capacity and memory bandwidth. This memory-centric bottleneck limits throughput and prevents a scalable, streaming-friendly hardware pipeline.

To address these constraints, we adopt an image partitioning strategy that processes the image in smaller patches. Partitioning reduces the memory capacity requirement by limiting the working set to a local region, and enables localized data reuse because intermediate results can be produced and consumed within the same patch before being written back. Moreover, a patch-based workflow naturally exposes a deterministic dataflow and regular memory access patterns, which are beneficial for SRAM banking and address generation in hardware.

However, a naive **direct partition**(ex: partition a 640×640 figure into four hundred 32×32 patches) may introduce **visible artifacts** near patch boundaries. This is because the illumination refinement relies on spatial gradients and neighborhood interactions; truncating the neighborhood at patch edges changes the effective constraints and leads to discontinuities after reconstruction. As shown in Fig. 2, direct partitioning tends to produce block-wise inconsistency, while an overlap-based partition (with a border/halo region) preserves boundary continuity by ensuring that local gradient operations and updates are computed with sufficient spatial context. Therefore, in the following subsections, we describe the adopted overlap-partition scheme and the corresponding memory organization that enables patch-wise processing while maintaining visual quality comparable to the non-partition baseline.

3.2. Overlap Partition

To avoid boundary padding and keep the patch grid aligned, we constrain the input image size to be compatible with the overlap-partition setting. Specifically, with patch size $P = 32$, valid size $V = 24$, and halo width $b = (P-V)/2 = 4$, the image height and width are chosen as

$$H = 2b + N \cdot V, \qquad W = 2b + M \cdot V, \tag{19}$$

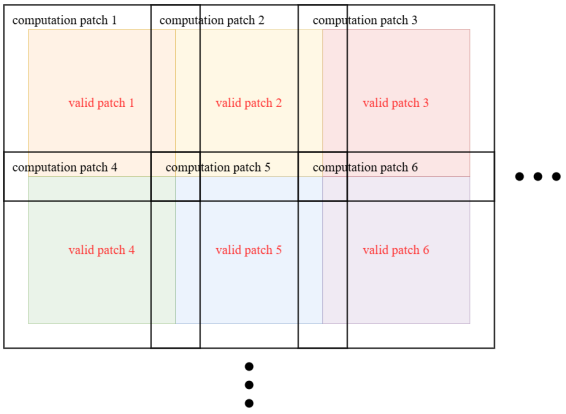

Figure 3. Overlap Partition Scheme

so that every 32×32 patch can be cropped with a stride $s = V$ without exceeding the image boundary. Since only the central valid region is stitched back and the outer b pixels on each side are discarded, the

(a) Original Image **(b)** Non Partition **(c)** Direct Partition **(d)** Overlap Partition

Figure 2. Image exposure correction by different methods

reconstructed output resolution becomes

$$H_{\text{out}} = H - 2b = N \cdot V, \qquad W_{\text{out}} = W - 2b = M \cdot V. \tag{20}$$

This design eliminates the need for explicit padding while preserving boundary continuity through overlap.

3.3. System Architecture

Fig. 4 illustrates the proposed dataflow for the patch-wise LIME solver. To minimize hardware cost while maintaining throughput, the computation is organized as a time-multiplexed pipeline with a small set of floating-point (FP64) functional units. Our design uses two FP64 multipliers, eight FP64 adders, and one reciprocal unit. Each FP multiplier supports either two independent real multiplications per cycle (dual-lane mode) or one complex multiplication per cycle (complex mode), allowing the same hardware to be reused across real-valued ALM updates and FFT-related complex operations. The eight FP adders are arranged as an 8-lane datapath and are reused across element-wise vector operations such as perform ∇, ∇^T, addition, subtraction. The reciprocal unit is shared by all divisions in the system. With this scheduling strategy, the ALM iteration is executed by repeatedly streaming data through SRAM banks and reusing functional units across subproblems, achieving a hardware-efficient implementation without allocating full-frame compute resources.

3.3.1. FFT and iFFT

In this stage, the module implements a 32-point (5-stage) 2D FFT/IFFT. The computation is performed in two steps: a row-wise FFT followed by a column-wise FFT. Due to SRAM access limitations, a transpose operation is inserted before the column-wise FFT, and the result is transposed back afterward to restore the original column order. Both FFT and IFFT adopt a Decimation-In-Time (DIT) architecture with the butterfly operation in the form of $a + b\omega$.

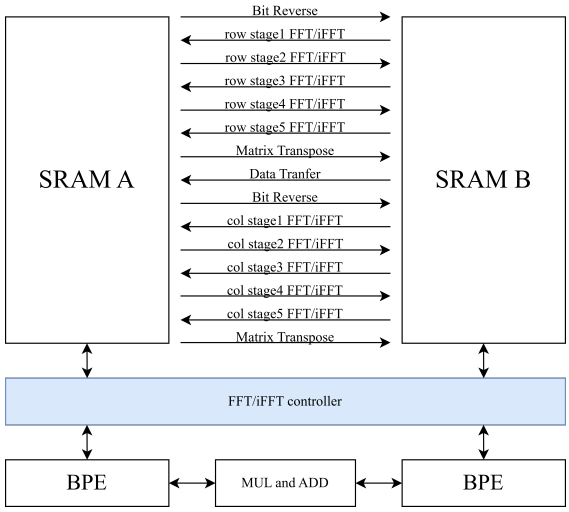

Figure 5. FFT/iFFT module

The data pairing remains identical for both FFT and IFFT, while

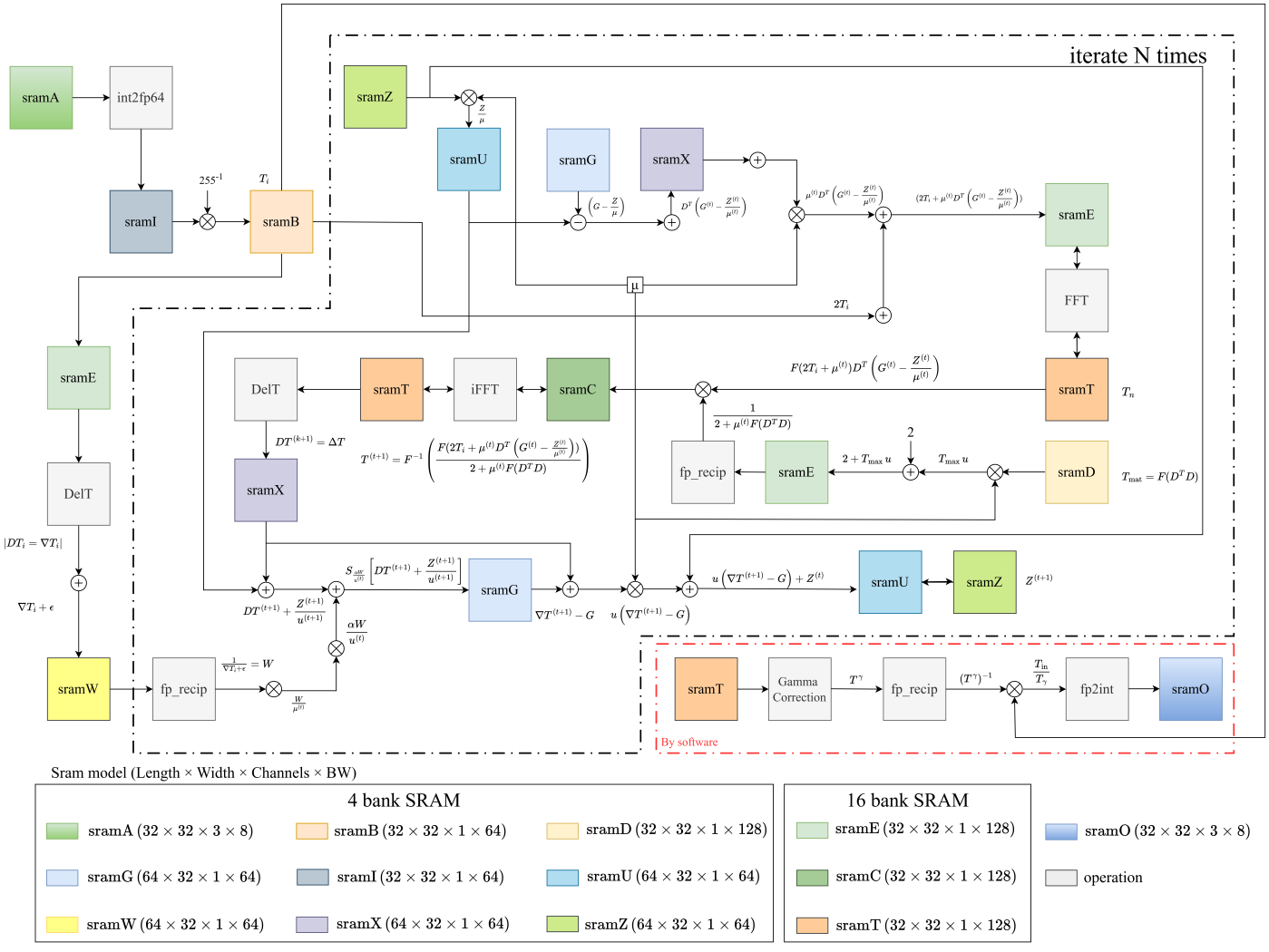

Figure 4. Dataflow System

the IFFT is realized by using the complex conjugates of the twiddle factors. When switching between FFT and IFFT modes, additional waiting cycles are required to ensure that the pipeline is fully flushed before mode transition. The architecture supports processing two butterfly pairs per cycle, allowing four data points to be accessed and written back to SRAM simultaneously. To fully utilize the computational units, a ping-pong buffer–based data flow is designed, enabling continuous operation without pipeline stalls. The SRAM is organized into 16 banks with 64 addresses per bank, providing a total storage capacity of 32×32 data points. Each memory location is 128 bits wide, which is used to store one complex data sample composed of real and imaginary FP64 values.

3.3.2. Floating Point Reciprocal

The reciprocal unit for IEEE 754 double-precision (FP64) numbers processes a 64-bit word comprising a 1-bit sign, an 11-bit exponent, and a 52-bit mantissa. To optimize hardware resources, special operands are mapped directly to constants via MUX logic, bypassing the iterative core:

Normal		Normal / Denormal ≈ Normal / 0
Denormal	Reciprocal ⟶	Normal / ± Inf ≈ ±Max Normal
±0		±Inf
±Inf		±0
NaN		NaN

For normalized cases $x = (-1)^{sign} \times (1.mantissa)_2 \times 2^{exponent-1023}$, the reciprocal results are determined by $sign_{recip} = sign$ and $exponent_{recip} = 2046 - exponent$. The mantissa is computed via the Goldschmidt division algorithm, implemented in a 4-stage pipeline using Q2.55 fixed-point format.

The refinement begins with an initial approximation F_0 from a 7-bit Look-Up Table (LUT) indexed by $mantissa[51 : 45]$, which outputs an 8-bit value representing the approximate reciprocal of the leading mantissa bits. The iterative refinement of the Goldschmidt division is performed as follows:

$$\frac{N_0}{D_0} = \frac{1}{mantissa}, \quad F_0 = LUT(mantissa[51:45])$$

$$\text{iteration 1} \implies \frac{N_1}{D_1} = \frac{N_0}{D_0} \times \frac{F_0}{F_0}, \quad F_1 = 2 - D_1$$

$$\text{iteration 2} \implies \frac{N_2}{D_2} = \frac{N_1}{D_1} \times \frac{F_1}{F_1}, \quad F_2 = 2 - D_2$$

$$\text{iteration 3} \implies \frac{N_3}{D_3} = \frac{N_2}{D_2} \times \frac{F_2}{F_2}$$

This method achieves quadratic convergence, where the error is squared in each iteration ($\epsilon_{i+1} = \epsilon_i^2$). By defining $D_i = 1 - \epsilon_i$, the refinement yields $D_{i+1} = 1 - \epsilon_i^2$. Three iterations effectively double the precision from 7 bits to 56 bits, exceeding the 52-bit FP64 requirement. A final round-to-nearest stage and exponent adjustment ensure a fully normalized result.

3.3.3. DelT operation

As illustrated in Fig. 7, we **do not** implement ∇T by explicitly performing the **matrix multiplications** in the original LIME formulation. Instead, we exploit the fact that ∇ is a structured finite-difference operator with wrap-around boundary conditions. By carefully interleaving SRAM address accesses, we fetch only the required neighboring pixels and perform tightly scheduled subtract operations to generate the horizontal and vertical differences. With this address-mapped implementation, DelT achieves the same effect as ∇T while significantly reducing computation and avoiding redundant data movement.

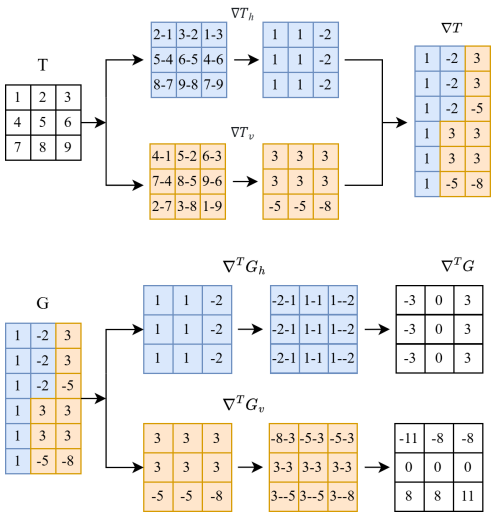

Figure 7. DelT operation

4. Expriment Result

Table 1. Performance 1

Syn Area	Allocate Area	Standard Cell Area	Core Utilization
876129	1682037	907396	53.9%

Table 2. Performance 2

Synthesis Timing	Post-Sim Timing
3.5ns	6.3ns

(a) Original Image **(b)** Software FP16 Enhanced Image **(c)** Software FP64 Enhanced Image **(d)** Hardware FP64 Enhanced Image

Figure 6. Comaparison between woftware and hardware result